Great Place To Work. Certified FEB 2023 – FEB 2024 INDIA

UPES

UNIVERSITY OF TOMORROW

nirf RANKING 2025

*RANKED 45 IN INDIA

#University Category

QS WORLD UNIVERSITY RANKINGS

NO.1 PVT. UNIVERSITY IN ACADEMIC REPUTATION IN INDIA

NAAC

ACCREDITED **GRADE 'A'** BY NAAC

PERFECT SCORE OF **150/150** AS A TESTAMENT TO EXECEPTIONAL E-LEARNING METHODS

UPES UNIVERSITY OF TOMORROW

Applied Machine Learning

Unit 02 – Lecture 02: Loss Functions

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Topic focus: **what number are we trying to make small?**

Repository: `https://github.com/tali7c/Applied-Machine-Learning`

Sources: Bishop, *Pattern Recognition and Machine Learning*, §1.5, §4.3, §7.1.;Zhang, Lipton, Li and Smola, *Dive into Deep Learning*, §3.4.;Shalev-Shwartz and Ben-David, *Understanding Machine Learning*, Ch. 15.;Murphy, *Probabilistic Machine Learning: An Introduction*, Ch. 5.

Quick Links

What a loss is | Regression | Classification | Margins | Choosing | Summary

Agenda

What a loss function is

Regression losses

Classification losses

Margin losses

Choosing a loss

Summary

How this lecture works

Every idea appears twice

1. **By hand** — small numbers, on paper, so you see the mechanism.
2. **In code** — the same thing, so you see the library does exactly that and nothing magic.

Commit before you look

Four slides ask you to predict an answer before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

Full derivations, more worked examples and 10 practice questions with solutions are in `notes.pdf`.

No Dataset, No Seed

The reproducibility convention, and why today is the exception

- Every other lecture fixes **one dataset and one seed**, always shown in the listing — the convention from Lecture 1.
- A loss function is arithmetic on numbers you already have. It does not sample, shuffle or initialise.
- So today there is **no dataset and no random number at all**: four house prices, three emails, six sensor readings, written out.

What that buys you

Every value in these slides is not just reproducible but *inevitable*. Run a listing and get something else, and it is a typing mistake — chance never got a turn. Randomness returns in Lecture 3, and with it `np.random.default_rng(0)` on every listing that needs one.

Where we are

Unit I gave us

The shape of the problem: data, a model family, and the need to generalise.

Unit II (today)

What quantity do we minimise?

Unit III (next)

How do we minimise it?

Why this order

Once you know what is being minimised and how the minimising is done, every method in Units V–VII is a variation on one theme rather than a new thing to memorise.

Fitting any supervised model: three steps

1. Propose a family of candidate models, controlled by parameters.

Fitting any supervised model: three steps

1. Propose a family of candidate models, controlled by parameters.
2. Define **one number** saying how badly a candidate does.

Fitting any supervised model: three steps

1. Propose a family of candidate models, controlled by parameters.
2. Define **one number** saying how badly a candidate does.
3. Search the parameters to make that number small.

Fitting any supervised model: three steps

1. Propose a family of candidate models, controlled by parameters.
2. Define **one number** saying how badly a candidate does.
3. Search the parameters to make that number small.

Step 2 is the loss function

That is this entire lecture. Step 3 is Unit III.

Fitting any supervised model: three steps

1. Propose a family of candidate models, controlled by parameters.
2. Define **one number** saying how badly a candidate does.
3. Search the parameters to make that number small.

Step 2 is the loss function

That is this entire lecture. Step 3 is Unit III.

The compression is where the judgement lives

A loss turns *a set of mistakes* into *one number*. By choosing how, you declare what kind of mistake you consider serious.

Three words that are not synonyms

Loss	computed on *one* example
Cost / objective	the loss averaged over the training set, plus regularization — the thing actually minimised
Metric	what you *report* to a human; need not be differentiable, need not be what you optimised

The loss and the metric are allowed to differ

Train with cross-entropy, report F1. Train with Huber, report RMSE. That is normal. What is *not* acceptable is failing to notice when they disagree about which model is better.

A loss must satisfy two things

1. Small when the model is right

Obvious — and it rules out less than you would think.

2. Usable by the optimiser

Differentiable almost everywhere, with a gradient that points somewhere useful.

Requirement 2 is why we cannot optimise the thing we actually care about.

We come back to this in the classification section.

The residual is the raw material

$$r_i = y_i - \hat{y}_i$$

Every regression loss is a rule for turning a residual into a non-negative penalty, then averaging.

MSE	$\frac{1}{n}\sum r_i^2$	squaring: twice as wrong costs *four* times
MAE	$\frac{1}{n}\sum \lvert r_i \rvert$	absolute: twice as wrong costs *twice*
RMSE	$\sqrt{\text{MSE}}$	same model as MSE — exists for *reporting*

RMSE is not a separate loss: the square root is increasing, so it gives the *identical* model. It exists so the number can be read aloud.

By hand: four houses

y (lakhs)	3.0	5.0	2.0	8.0
$\hat{y}$	2.5	5.5	4.0	7.0
r	0.5	-0.5	-2.0	1.0
r^2	0.25	0.25	**4.0**	1.0
$\lvert r \rvert$	0.5	0.5	**2.0**	1.0

$$\text{MSE} = \tfrac{5.5}{4} = 1.375 \qquad \text{MAE} = \tfrac{4}{4} = 1.0 \qquad \text{RMSE} = \sqrt{1.375} = 1.173$$

Look at the third house

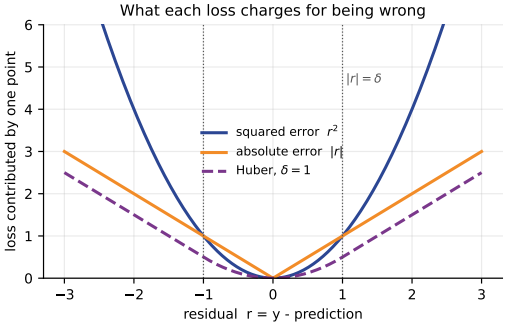
It contributes **4 of the 5.5** squared-error total — 73% of the loss from 25% of the data. Under MAE: 2 of 4, exactly half.

The same thing in code

```
import numpy as np
from sklearn.metrics import mean_squared_error, mean_absolute_error

y = np.array([3.0, 5.0, 2.0, 8.0])
yhat = np.array([2.5, 5.5, 4.0, 7.0])
r = y - yhat

print("MSE np ", np.mean(r**2))
print("MAE np ", np.mean(np.abs(r)))
print("MSE sklearn", mean_squared_error(y, yhat))
```

```
MSE  np      1.375
MAE  np      1.0
MSE  sklearn 1.375
```

What each loss charges



A bowl, a V with a kink at zero, and Huber — bowl near zero, V far away.

Predict first #1

Six readings, and you must predict one constant

```
[10, 11, 12, 13, 14, 14]
```

A sensor now corrupts the last one: 14 becomes 94.

What happens to the best constant under MSE? Under MAE?

Predict first #1

Six readings, and you must predict one constant

```
[10, 11, 12, 13, 14, 14]
```

A sensor now corrupts the last one: 14 becomes 94.

What happens to the best constant under MSE? Under MAE?

	best under MSE	**best under MAE**
clean	12.33	12.50
with outlier	**25.67**	**12.50**

Predict first #1

Six readings, and you must predict one constant

```
[10, 11, 12, 13, 14, 14]
```

A sensor now corrupts the last one: 14 becomes 94.

What happens to the best constant under MSE? Under MAE?

	best under MSE	**best under MAE**
clean	12.33	12.50
with outlier	**25.67**	**12.50**

The MSE answer moved *past every clean observation*. MAE did not move.

Why: MSE fits the mean, MAE fits the median

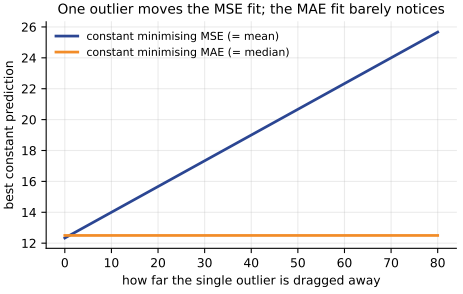
Minimising $\sum(y_i - c)^2$

gives $c = \bar{y}$, the **mean**

Minimising $\sum |y_i - c|$

gives $c = \text{median}(y)$

Both proofs are in the notes.



The sentence to remember

Choosing a loss is not choosing how to *score* your model. It is choosing **what your model will become**.

Confirming it numerically

```
outlier = np.array([10., 11., 12., 13., 14., 94.])
grid = np.linspace(0, 100, 200001)
mse = ((outlier[:,None] - grid[None,:])**2).mean(axis=0)
mae = (np.abs(outlier[:,None] - grid[None,:])).mean(axis=0)
print("argmin MSE ->", round(grid[mse.argmin()], 4))
print("argmin MAE ->", round(grid[mae.argmin()], 4))
```

```
argmin MSE -> 25.6665
argmin MAE -> 12.0
```

Confirming it numerically

```
outlier = np.array([10., 11., 12., 13., 14., 94.])
grid = np.linspace(0, 100, 200001)
mse = ((outlier[:,None] - grid[None,:])**2).mean(axis=0)
mae = (np.abs(outlier[:,None] - grid[None,:])).mean(axis=0)
print("argmin MSE ->", round(grid[mse.argmin()], 4))
print("argmin MAE ->", round(grid[mae.argmin()], 4))
```

```
argmin MSE -> 25.6665
argmin MAE -> 12.0
```

Why 12.0 and not the median 12.5?

With an *even* sample the MAE is **flat** across $[12, 13]$ — every constant there is equally optimal, and `argmin` returns the first. A flat loss means **zero gradient**: that is why MAE is harder to optimise than MSE.

Huber: take the good half of each

$$L_\delta(r) = \begin{cases} \frac{1}{2} r^2, & |r| \le \delta \\ \delta\left(|r| - \frac{1}{2}\delta\right), & |r| > \delta \end{cases}$$

The pieces meet smoothly

The linear branch is the tangent to the quadratic at $r = \delta$, so value *and* slope match. Differentiable everywhere.

δ carries the units of y

$\delta = 1$ means something different for prices in lakhs than for temperatures. Set it by cross-validation — never copy it from a tutorial.

By hand, then in code: Huber with $\delta = 1$

Same four residuals: $0.5,\ -0.5,\ -2.0,\ 1.0$

$\lvert 0.5\rvert \leq 1$	quadratic	$\frac{1}{2}(0.5)^2 = 0.125$
$\lvert -0.5\rvert \leq 1$	quadratic	0.125
$\lvert -2.0\rvert > 1$	**linear**	$1 \cdot (2 - 0.5) = 1.5$
$\lvert 1.0\rvert \leq 1$	quadratic	$\frac{1}{2}(1)^2 = 0.5$
mean		$2.25/4 = \mathbf{0.5625}$

Squared error charged that third point **4.0**. Huber charged it **1.5**.

Huber in code

```
def huber(r, d=1.0):
    a = np.abs(r)
    return np.where(a <= d, 0.5*r**2, d*(a - 0.5*d))

print("per point", huber(r, 1.0))
print("mean ", huber(r, 1.0).mean())
```

```
per point [0.125 0.125 1.5   0.5  ]
mean      0.5625
```

δ larger than every residual $\Rightarrow$ you have chosen scaled MSE.

The obvious loss, and why it fails

Charge **1** for a wrong answer, **0** for a right one.

This is the **0–1 loss** — exactly what accuracy measures.

It is also almost useless for *training*.

The next slide asks you to see why before I say it.

Predict first #2

A one-parameter classifier predicts class 1 when $wx > 0$. Four points, two per class. We sweep w.

How many distinct values can accuracy take? Can it tell $w{=}1$ from $w{=}2$?

Predict first #2

A one-parameter classifier predicts class 1 when $wx > 0$. Four points, two per class. We sweep w.

How many distinct values can accuracy take? Can it tell $w{=}1$ from $w{=}2$?

```
w= -2.0  accuracy=0.00  log-loss=3.0725
w= -1.0  accuracy=0.00  log-loss=1.7201
w=  0.0  accuracy=0.50  log-loss=0.6931
w=  1.0  accuracy=1.00  log-loss=0.2201
w=  2.0  accuracy=1.00  log-loss=0.0725
```

Predict first #2

A one-parameter classifier predicts class 1 when $wx > 0$. Four points, two per class. We sweep w.

How many distinct values can accuracy take? Can it tell $w{=}1$ from $w{=}2$?

```
w= -2.0  accuracy=0.00  log-loss=3.0725
w= -1.0  accuracy=0.00  log-loss=1.7201
w=  0.0  accuracy=0.50  log-loss=0.6931
w=  1.0  accuracy=1.00  log-loss=0.2201
w=  2.0  accuracy=1.00  log-loss=0.0725
```

Three values. And no, it cannot.

Accuracy is a **step function**: flat everywhere, derivative zero. Gradient descent standing on it is told "every direction is equally good". Cross-entropy falls strictly the whole way.

Surrogate losses

The move that makes classification trainable

We optimise a **surrogate** — a smooth loss that stands in for the thing we actually care about.

What we care about	0–1 loss (accuracy)
What we minimise	cross-entropy, or hinge

This is why the number you optimise and the number you report are so often different. Not carelessness — necessity.

Binary cross-entropy

$$\mathrm{BCE} = -\frac{1}{n}\sum_i \Big[\, y_i \log p_i + (1 - y_i)\log(1 - p_i) \,\Big]$$

The bracket has two terms but only ever contributes one

$y_i = 1 \Rightarrow$ second term vanishes. $\quad y_i = 0 \Rightarrow$ first term vanishes.

So the formula says something very simple

Take the probability the model gave **the class that was actually correct**. Take its negative logarithm. Average. That is all it is.

By hand: three emails

true y ($1 =$ spam)	1	0	1
model p (spam)	0.9	0.2	0.4
prob. of *correct* class	0.9	**0.8**	0.4
$-\log(\cdot)$	0.1054	0.2231	**0.9163**

Row three is the only step with content: email 2's correct class was *not* spam, so we take $1 - p = 0.8$.

$$\mathrm{BCE} = \frac{0.1054 + 0.2231 + 0.9163}{3} = \mathbf{0.4149}$$

The third email was not even confidently wrong — 0.4 is near a coin flip — and it still cost more than the other two combined.

The same thing in code

```
from sklearn.metrics import log_loss

y = np.array([1, 0, 1])
p = np.array([0.9, 0.2, 0.4])
pc = np.where(y == 1, p, 1 - p) # prob given to the correct class

print("prob correct ", pc)
print("BCE by hand ", (-np.log(pc)).mean())
print("BCE sklearn ", log_loss(y, p))
```

```
prob correct  [0.9 0.8 0.4]
BCE by hand   0.414931599615397
BCE sklearn   0.414931599615397
```

Predict first #3

A model gives probability **0.99** to the correct class. Loss $= 0.0101$.

Another gives 0.01 to the correct class. How many times worse?

Predict first #3

A model gives probability **0.99** to the correct class. Loss $= 0.0101$.

Another gives 0.01 to the correct class. How many times worse?

p correct	loss	
0.99	0.0101	confident and right — nearly free
0.50	0.6931	a coin flip costs $\log 2$
0.01	**4.6052**	**458**$\times$ the cost of being right
0.001	6.9078	and it keeps going

Predict first #3

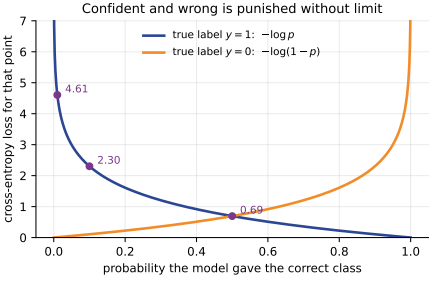
A model gives probability **0.99** to the correct class. Loss $= 0.0101$.

Another gives 0.01 to the correct class. How many times worse?

p correct	loss	
0.99	0.0101	confident and right — nearly free
0.50	0.6931	a coin flip costs $\log 2$
0.01	**4.6052**	**458**$\times$ the cost of being right
0.001	6.9078	and it keeps going

As $p \to 0$ the loss grows **without bound**. That is the design: it forces models to be honest about uncertainty instead of confidently guessing.

Confident and wrong



$\log 0$ is not a number

Exactly 0 or 1 gives infinite loss and kills the run. Libraries clip into $[\varepsilon, 1-\varepsilon]$ — which is why you feed `log_loss` *probabilities*, not hard 0/1 predictions.

Categorical vs sparse categorical: the same loss

```
P = np.array([[0.7,0.2,0.1], [0.1,0.8,0.1], [0.2,0.2,0.6]])
onehot = np.array([[1,0,0], [0,1,0], [0,0,1]]) # categorical
sparse = np.array([0, 1, 2]) # sparse categorical

print("categorical", -np.sum(onehot*np.log(P), axis=1).mean())
print("sparse ", -np.log(P[np.arange(3), sparse]).mean())
```

```
categorical 0.3635480396729776
sparse      0.3635480396729776
```

Identical to sixteen decimals — it is the same arithmetic. **Only the label format differs.**

Why the sparse form exists

Pure memory economics

$K = 10{,}000$ classes, batch of 256.

One-hot: 2.56 million numbers, of which **256** are non-zero.
Sparse: **256 integers**, indexed directly.

One of the commonest beginner errors

Choosing the wrong one in Keras or PyTorch gives an unhelpful error message.

Check the shape of your labels: 2-D means categorical, 1-D means sparse.

When there is no probability to work with

An SVM outputs a raw score $f(x)$ — positive for one class, negative for the other. Cross-entropy needs a probability, so we need something else.

Relabel classes as $y \in \{-1, +1\}$ and define the **margin**

$$m_i = y_i \, f(x_i)$$

Positive exactly when the prediction is *correct*; its size says how *decisively*. A margin loss looks at nothing else.

Hinge loss

$$L_{\text{hinge}}(m) = \max(0,\ 1 - m)$$

Margin at least $1 \Rightarrow$ loss zero. Otherwise pay the shortfall.

The consequence worth remembering

Hinge **stops caring** once you are right by a comfortable margin. Cross-entropy never entirely stops — it will always take a little more confidence.

That is why an SVM is determined by a handful of *support vectors*, while logistic regression is influenced by every point.

Predict first #4

true y	$+1$	-1	$+1$
score $f(x)$	2.0	-0.5	0.3

All three are classified correctly. So is the total hinge loss zero?

Predict first #4

true y	$+1$	-1	$+1$
score $f(x)$	2.0	-0.5	0.3

All three are classified correctly. So is the total hinge loss zero?

margin $m = yf$	2.0	0.5	0.3
$\max(0, 1 - m)$	**0.0**	0.5	0.7

$$L = (0.0 + 0.5 + 0.7)/3 = \mathbf{0.4}$$

Predict first #4

true y	$+1$	-1	$+1$
score $f(x)$	2.0	-0.5	0.3

All three are classified correctly. So is the total hinge loss zero?

margin $m = yf$	2.0	0.5	0.3
$\max(0, 1 - m)$	**0.0**	0.5	0.7

$$L = (0.0 + 0.5 + 0.7)/3 = \mathbf{0.4}$$

Two *correctly classified* points still cost — they sit inside the margin band. That is what produces a **wide** boundary.

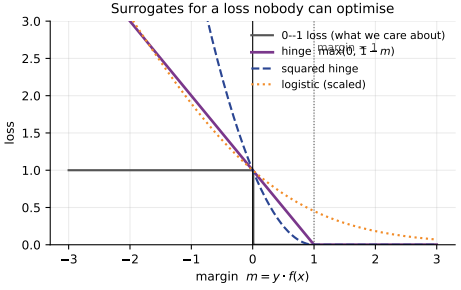
Hinge in code — and a floating-point warning

```
from sklearn.metrics import hinge_loss

ys = np.array([1, -1, 1])
f = np.array([2.0, -0.5, 0.3])

print("margins ", ys * f)
print("hinge by hand", np.maximum(0, 1 - ys*f).mean())
print("hinge sklearn", hinge_loss(ys, f))
```

```
margins       [2.  0.5 0.3]
hinge by hand 0.39999999999999997
hinge sklearn 0.39999999999999997
```

That is 0.4. Binary floating point cannot represent it exactly — never test loss values with ==; use `np.isclose`.

The family portrait



- Every surrogate sits **above** the 0–1 step — that is what makes it a legitimate stand-in.
- Hinge reaches exactly zero at $m = 1$; logistic approaches but never arrives.
- Left of zero, hinge grows linearly and squared hinge quadratically.

Triplet loss: a different question entirely

Sometimes we do not want to classify — we want a **representation** where similar things are close. Face recognition: you cannot have one output per person when new people keep arriving.

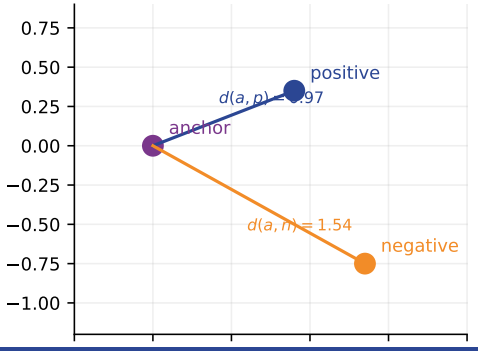
Three examples at a time

anchor a
positive p — same identity
negative n — different identity

$$L = \max\left(0,\ d(a,p) - d(a,n) + \alpha\right)$$

The anchor should be closer to the positive by at least α.

Triplet loss $= \max(0,\ d(a,p) - d(a,n) + \alpha)$
$= \max(0,\ 0.97 - 1.54 + 1.0) = 0.42$



Most triplets teach you nothing

For the figure: $L = \max(0,\ 0.97 - 1.54 + 1.0) = \mathbf{0.42}$

The ordering is already right, but only by 0.58 — short of the margin — so a penalty remains.

Triplet mining

Pick three examples at random and the constraint is usually *already satisfied*: zero loss, zero gradient, nothing learned.

Real systems spend most of their engineering effort hunting the hard cases where $d(a, p)$ is nearly as large as $d(a, n)$. Without mining, training stalls almost immediately.

A decision table

Situation	Use	Because
Regression, clean, symmetric errors	MSE	smooth, fast; the mean is the sensible target
Regression, outliers you distrust	MAE	fits the typical case
Regression, outliers you cannot exclude	Huber	quadratic where it matters, linear where it would distort
Two classes, want probabilities	Binary cross-entropy	calibrated; punishes confident errors
K classes, one-hot labels	Categorical CE	same loss, one-hot API
K classes, integer labels	Sparse categorical CE	same loss, far less memory
Two classes, want a wide boundary	Hinge	ignores points already safely correct
Embedding where similar = close	Triplet	relative distance; handles unseen identities

The mistake to avoid

Do not choose by asking which gives the smaller number

Losses live on different scales and are **not comparable to each other**.

From our four houses — *identical predictions*:

$$\text{MAE} = 1.0 \qquad \text{MSE} = 1.375$$

MAE is not "better". They are different quantities in different units.

The rule

Compare **models** using a single fixed metric on held-out data.
Use the loss to *train*; use the metric to *decide*.

Summary

1. A loss compresses mistakes into one number; that choice declares what kind of mistake you consider serious.
2. Loss, cost and metric are three different things.
3. **MSE fits the mean, MAE fits the median.** One outlier moved our MSE constant from 12.33 to 25.67 and left MAE untouched.
4. Huber is MSE near zero and MAE far away. δ carries the units of y.
5. Accuracy cannot be a training loss — step function, zero gradient. We minimise smooth surrogates.
6. Cross-entropy $= -\log$ (probability given to the correct class). Confident and wrong is unboundedly expensive.
7. Categorical and sparse categorical are the *same loss*, different label format.
8. Hinge charges nothing beyond margin 1 — hence support vectors.
9. Triplet loss trains on relative distance and needs hard mining.

Before the next lecture

Read

`notes.pdf` — full derivations, the median proof, and **10 practice questions with worked solutions**.

Do

Questions 2, 4 and 10. Question 10 ties the whole lecture together.

Next: Unit III — Optimizers

We now know *what* to minimise.

Next: gradient descent, mini-batches and momentum — *how* the minimising actually happens.

Materials: `https://github.com/tali7c/Applied-Machine-Learning`

Sources: Bishop, *Pattern Recognition and Machine Learning*, §1.5, §4.3, §7.1.;Zhang, Lipton, Li and Smola, *Dive into Deep Learning*, §3.4.;Shalev-Shwartz and Ben-David, *Understanding Machine Learning*, Ch. 15.;Murphy, *Probabilistic Machine Learning: An Introducti[illegible]6.